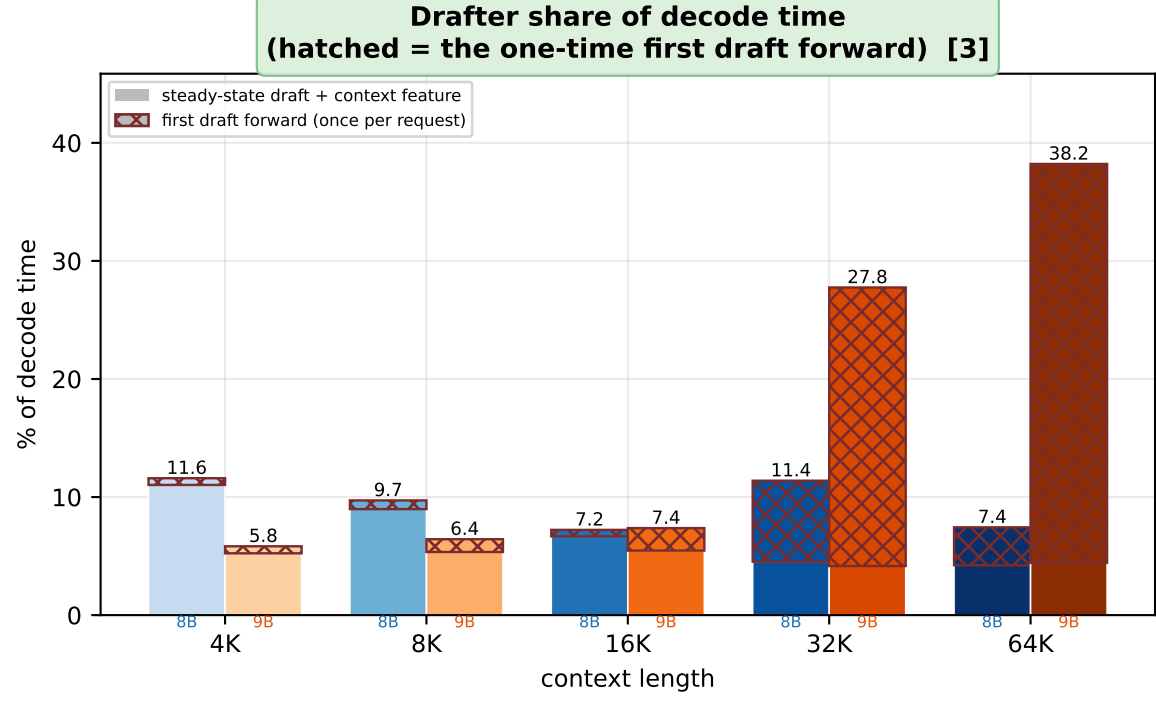
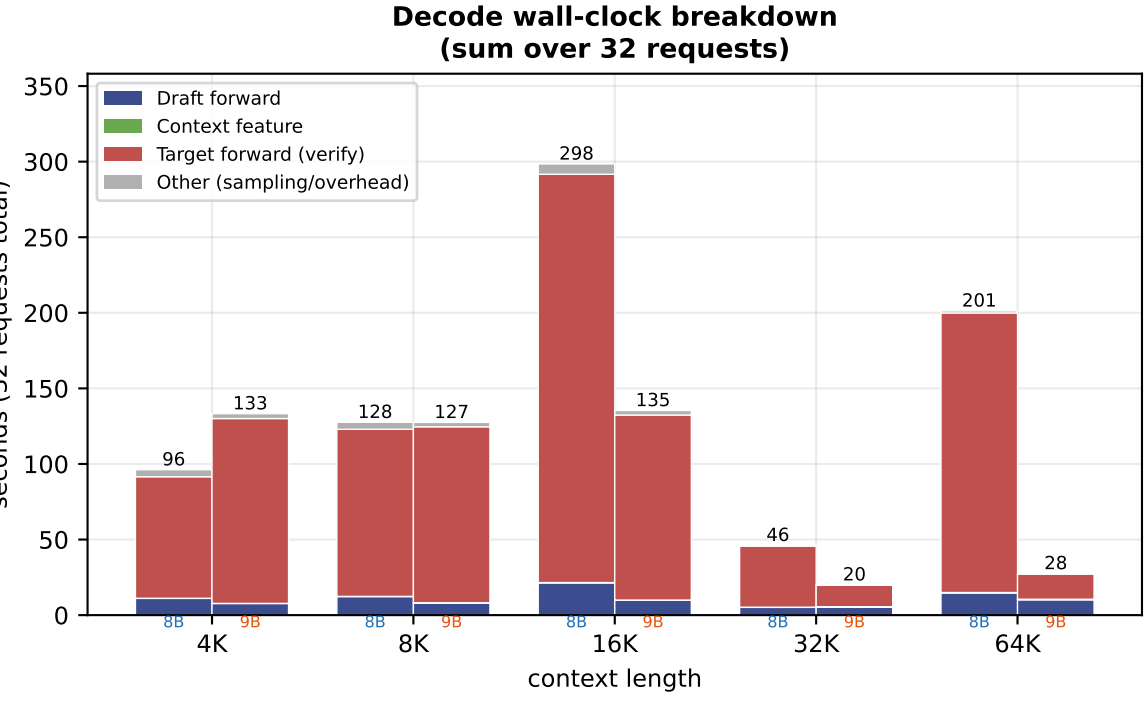
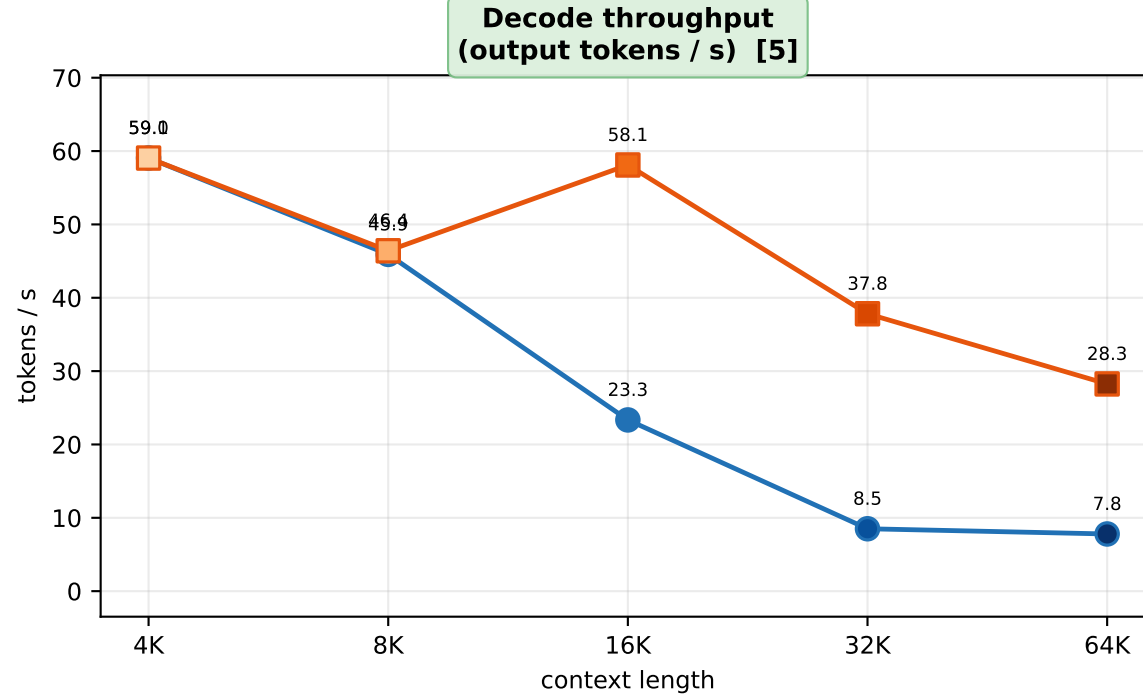
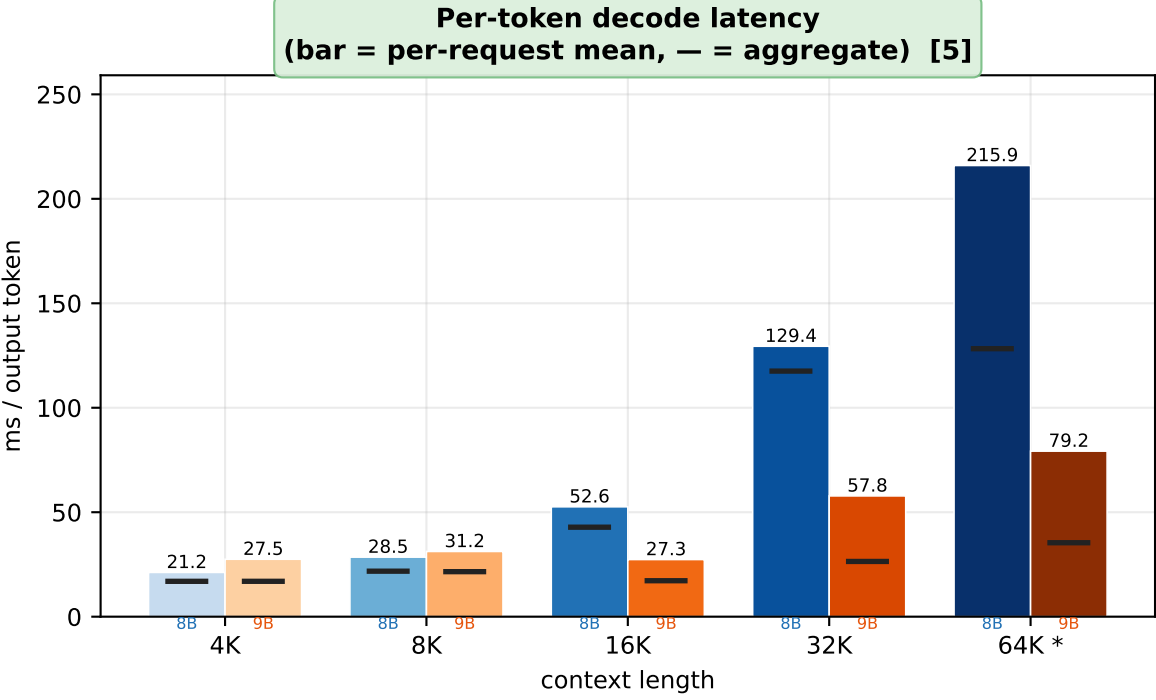
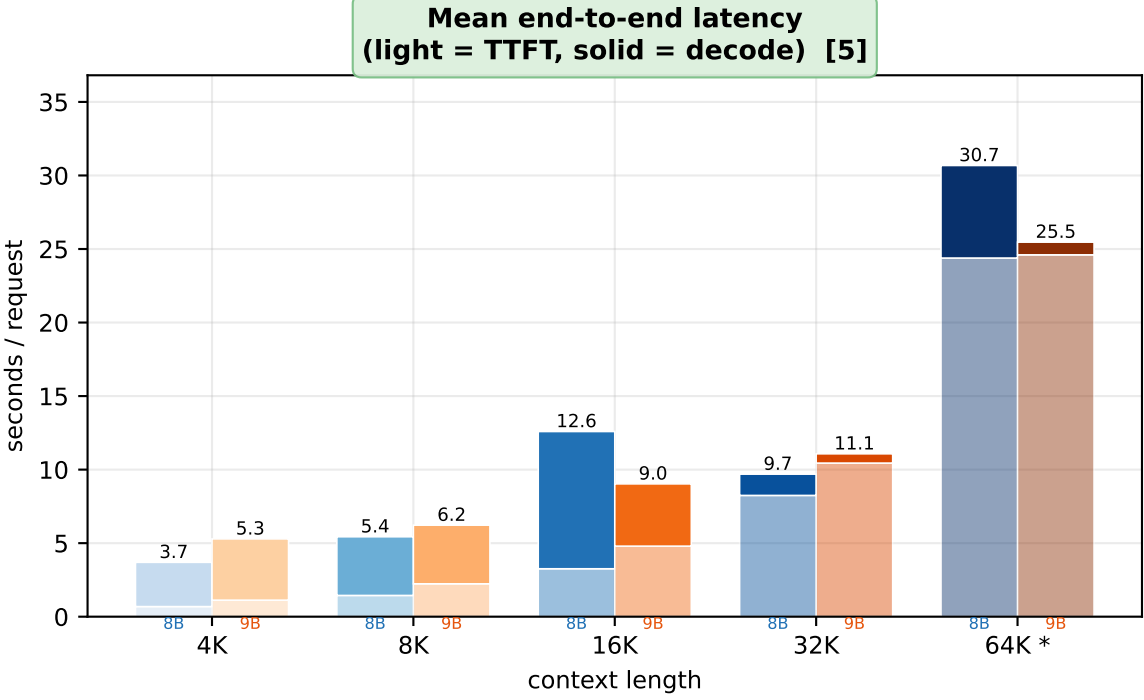
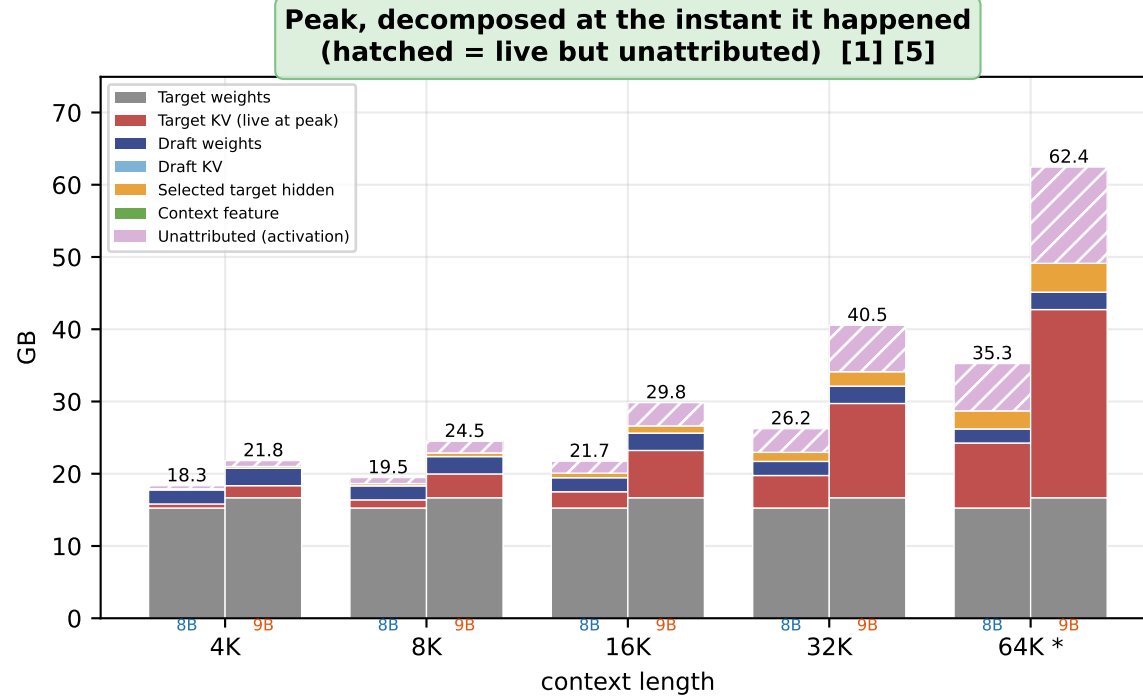
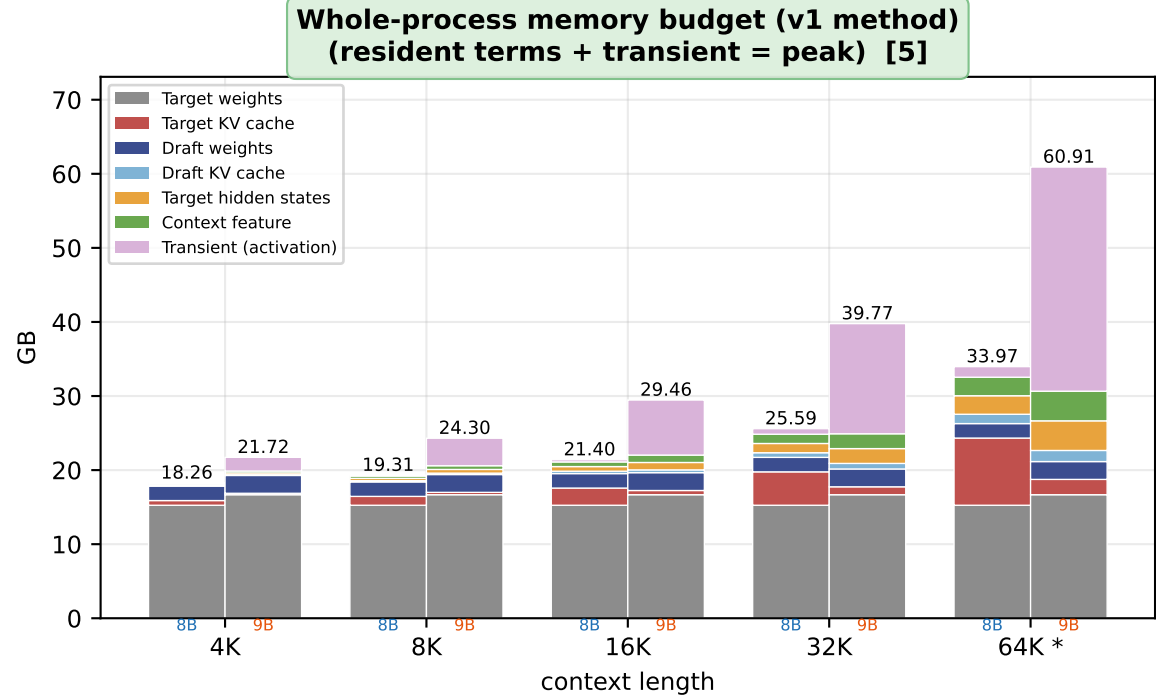
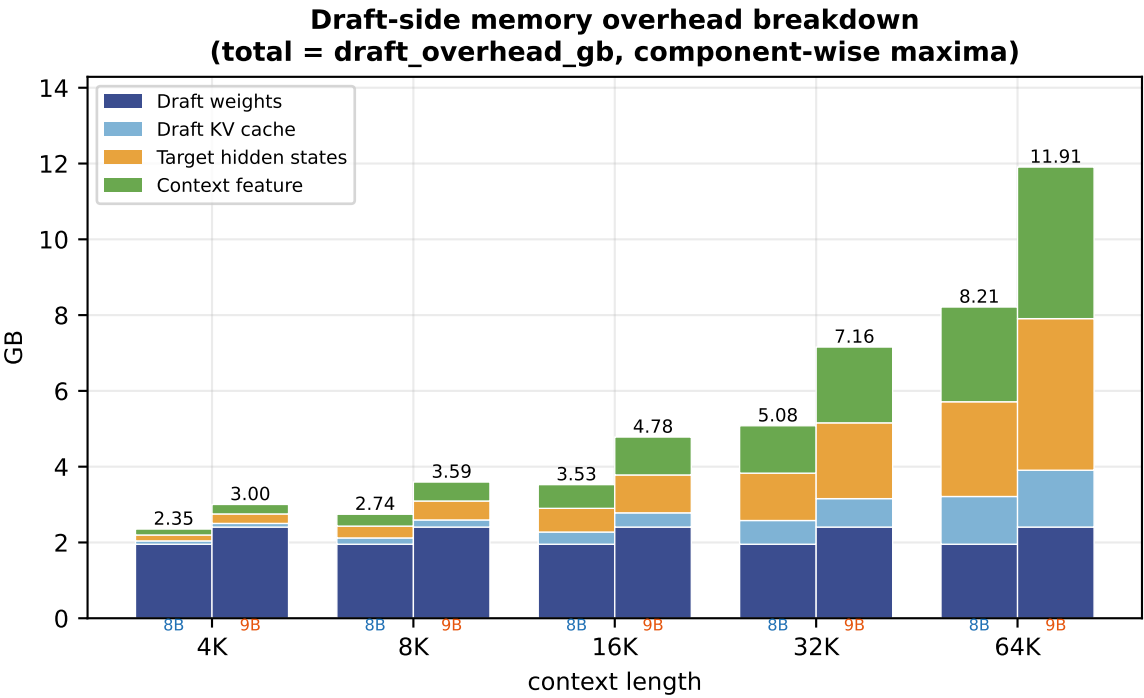
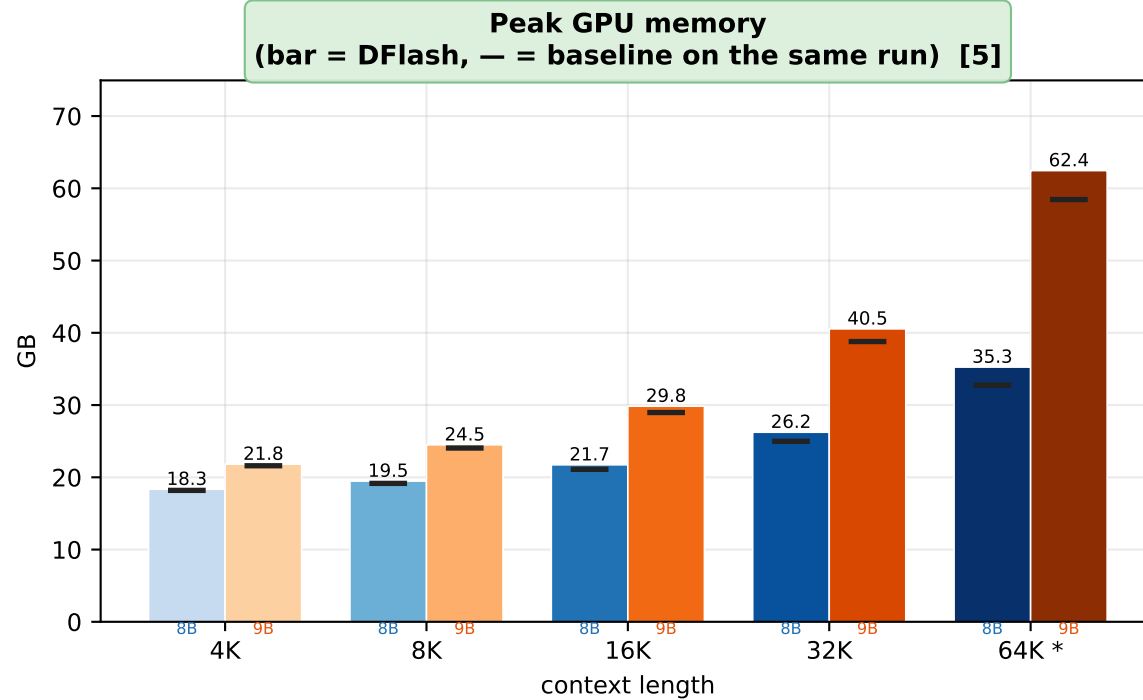
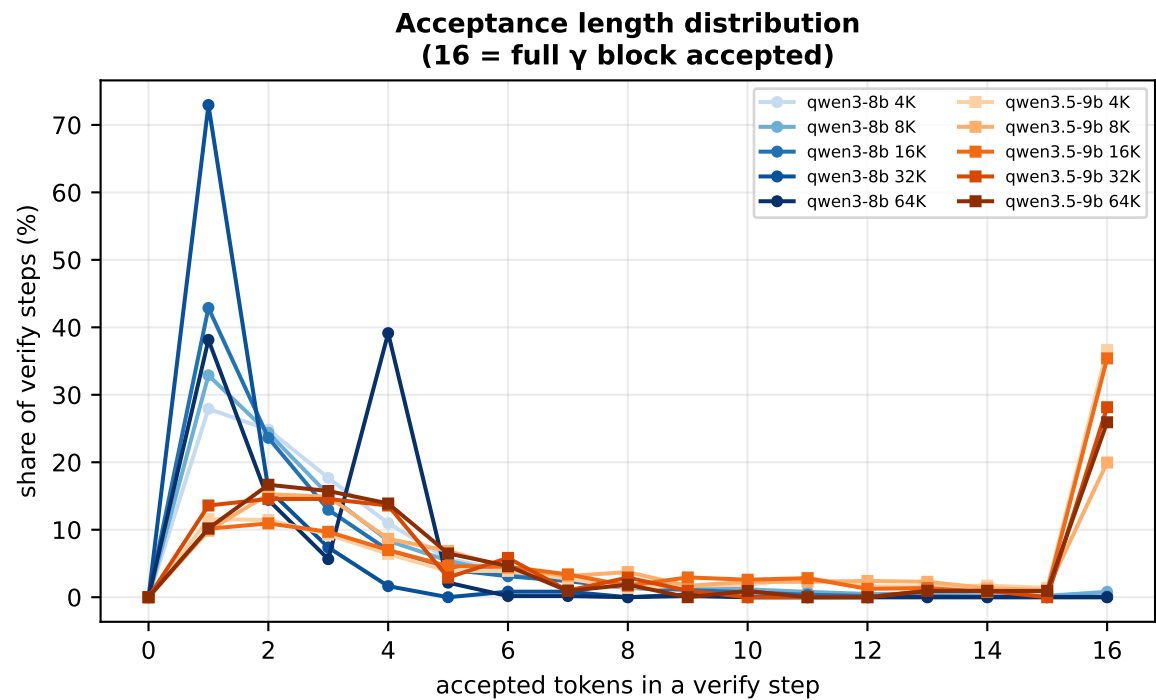
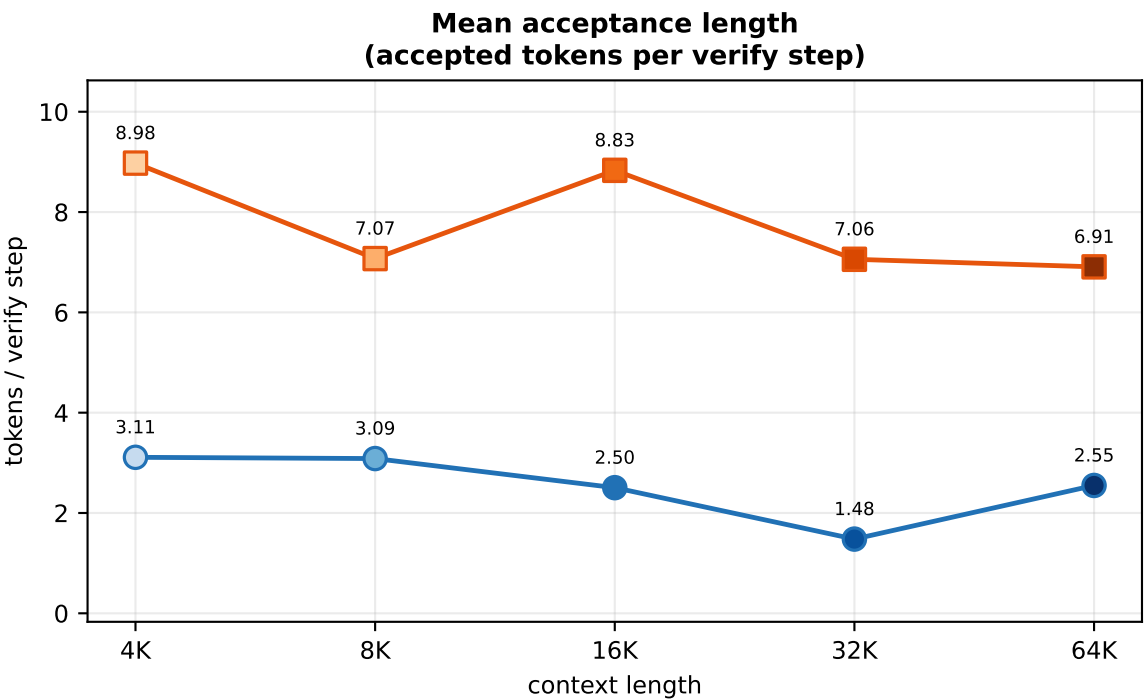


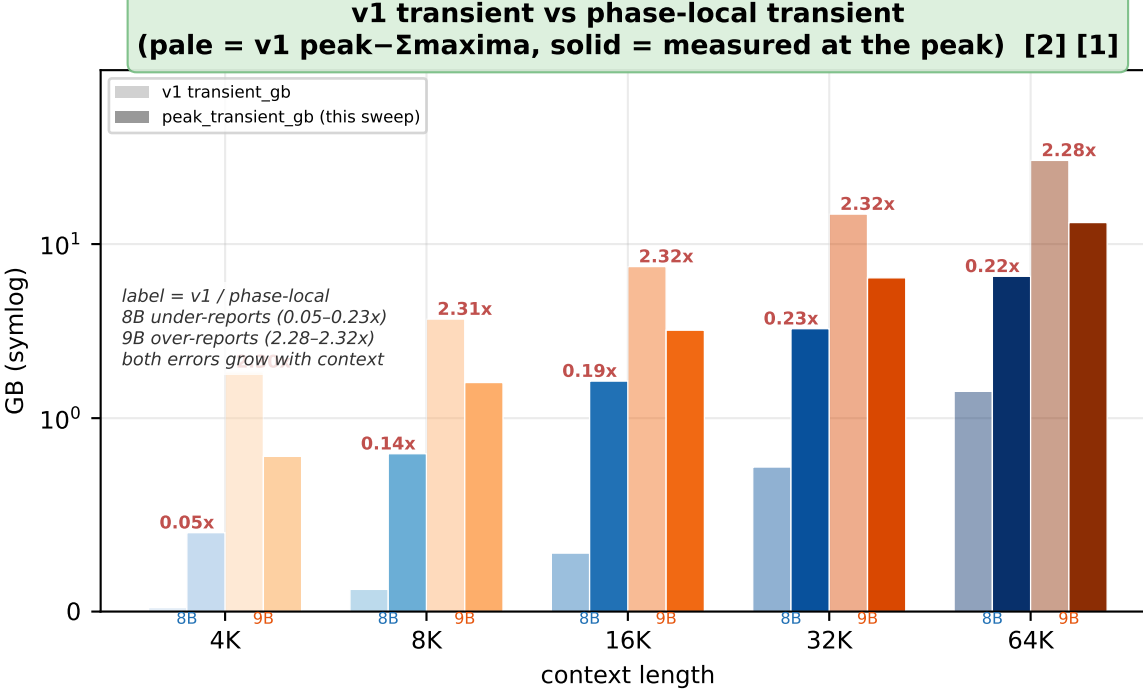
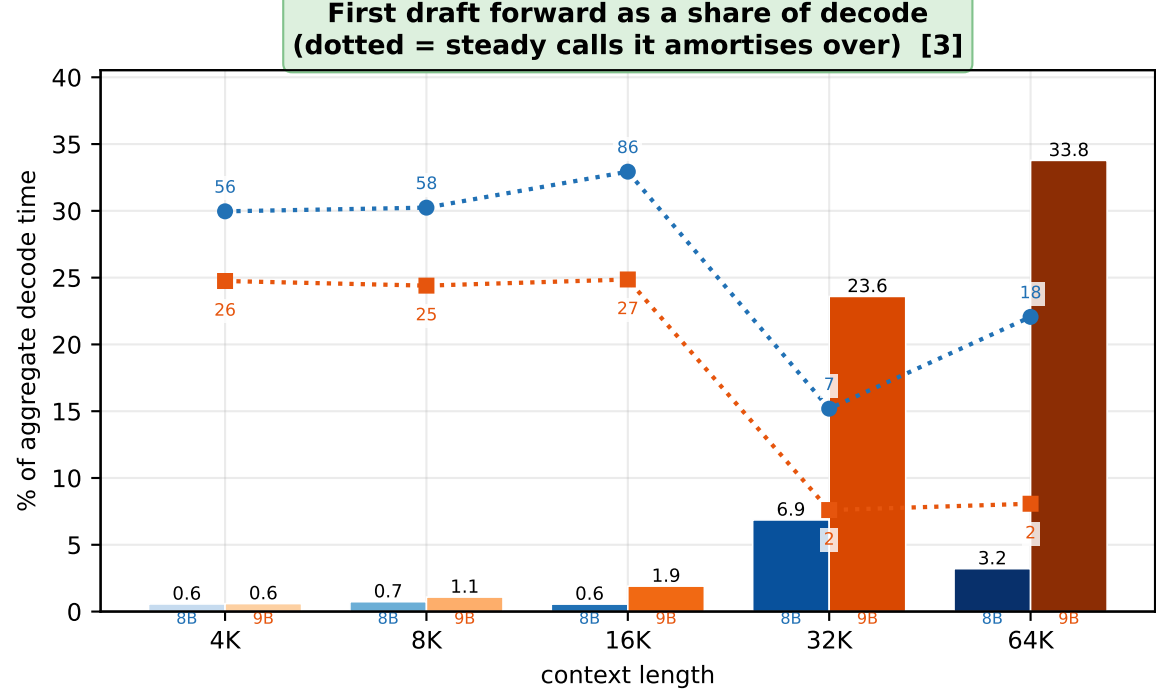
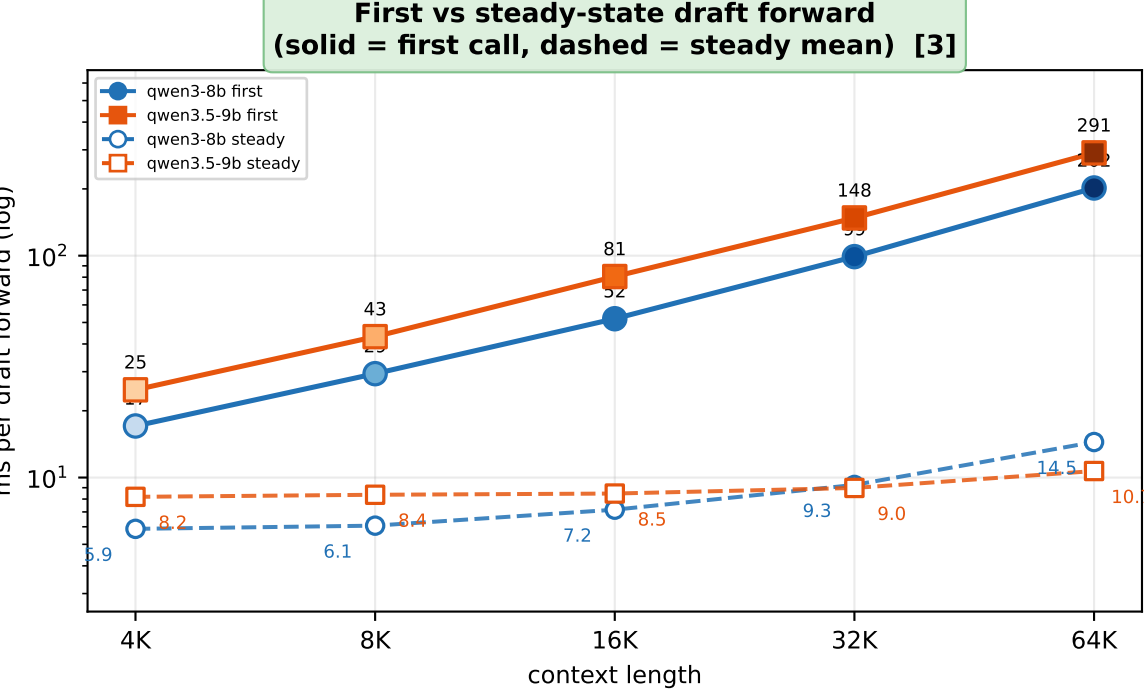
DFlash speculative decoding — LongBench-E averages, hidden_states=selective, phase-instrumented sweep
Qwen3-8B vs Qwen3.5-9B @ 4K / 8K / 16K / 32K / 64K · block=16 · gamma=15 · greedy



Which phase set the peak
(peak_phase, and DFlash's live share of it) [4] [1]

run	phase holding the peak	interval	DFlash@pk
8B 4K	prefill: target forward	18.3G	2.11G
8B 8K	prefill: target forward	19.5G	2.27G
8B 16K	prefill: target forward	21.7G	2.58G
8B 32K	prefill: target forward	26.2G	3.28G
8B 64K	prefill: target forward	35.3G	4.45G
9B 4K	prefill: target forward	21.8G	2.66G
9B 8K	prefill: target forward	24.5G	2.91G
9B 16K	prefill: target forward	29.8G	3.41G
9B 32K	prefill: target forward	40.5G	4.41G
9B 64K	prefill: target forward	62.4G	6.41G

Unanimous across all ten runs: the peak is a target prefill layer, never a DFlash allocation. Of the DFlash column, draft KV and the context feature are 0.00 GB at that instant — both are built after prefill ends. What is live is the draft weights plus the selected target hidden, and the latter is exactly the DFlash-minus-baseline peak gap (panel 3): 0.16 GB at 4K rising to 2.50 / 4.00 GB at 64K.



qwen3-8b · 4K qwen3-8b · 16K qwen3-8b · 64K qwen3.5-9b · 8K qwen3.5-9b · 32K qwen3-8b (trend) qwen3.5-9b (trend)
qwen3-8b · 8K qwen3-8b · 32K qwen3.5-9b · 4K qwen3.5-9b · 16K qwen3.5-9b · 64K

Samples per context length [8B / 9B where they differ] — 4K: n=32 (13 datasets) · 8K: n=32 (12 datasets) · 16K: n=32 (10 / 9 datasets) · 32K: n=32 (9 datasets, 31 composed) · 64K: n=32 (8 datasets, 32 composed)
Mean output tokens per request [8B / 9B] — 4K: 178 / 246 · 8K: 183 / 185 · 16K: 218 / 246 · 32K: 12 / 24 · 64K: 49 / 24 (long-context runs emit far fewer tokens; compare per-token metrics, not per-request ones)

Panels highlighted in green are new or re-measured relative to plot_summary.py (v1, same records directory, pre-instrumentation sweep) — ADDED = a quantity the v1 records did not carry, REVISED = same quantity, measured at a different instant

- [1] ADDED summary.phase_memory, a new record field. The run is cut into twelve named phases and each one records allocated-before, allocated-after and its own interval peak, plus the live component split probed at the occurrence that peaked. peak_phase names the phase holding the largest interval peak.
- [2] REVISED Transient is now peak_transient_gb -- the peak phase's interval peak minus what that phase kept resident, read at one instant. The v1 transient_gb was peak minus a sum of component-wise maxima taken at different instants, so it was not a decomposition: it under-reports on Qwen3-8B and over-reports on Qwen3.5-9B, and both errors grow linearly with context. Panel 15 plots the gap.
- [3] ADDED mean_first_draft_forward_s / mean_steady_draft_forward_s and the matching cache and stage fields, all new. The drafter's first forward projects the whole context into its KV; every later one appends a block. v1 averaged the two together, which charged the one-time build to every call.
- [4] REVISED Where the peak landed is now read from phase_memory rather than peak_site alone. peak_site names the operation; peak_phase names the phase and comes with the component split live at that instant, which is what separates DFlash's contribution to the peak from the target's.
- [5] REVISED Device count differs from the full-mode records: 8B/64K 2->1, 9B/32K 2->1, 9B/64K 3->2. Selective frees enough memory to fit on fewer cards. 9B/64K is still sharded and starred on the x axis, so its timings are pipeline-parallel. Memory stays comparable across a device change; timings do not -- going from sharded to single-GPU speeds the "baseline" up more than it speeds DFlash, so a speedup can read lower without DFlash having got slower.
- [6] REVISED 9B/64K is sharded over more than one card (starred on the x axis). Per-stage draft timings are recorded with CUDA events on the drafter's stream, so a stage whose weights live on the other card -- lm_head, in practice -- is timed as the launch, not the work: it reads as ~0.05 ms against ~2.8 ms on a single card. Read the stage split of a starred column qualitatively only, and never subtract it from a single-GPU column.